
REPASO: BÁSICOS DE ML

Fernanda Sobrino

2026

Aprendizaje Supervisado

Aprendizaje supervisado

Recordemos:

- ▶ Tenemos inputs x y outputs y
- ▶ Queremos aprender una función que mapee inputs a outputs
- ▶ Esta función depende de un conjunto de parámetros
- ▶ Entrenar el modelo significa encontrar parámetros que produzcan buenas predicciones sobre los datos

$$\hat{y} = f(x, \phi)$$

Notación

- ▶ Input: x , independientemente de su dimensión
- ▶ Output: y
- ▶ Parámetros: ϕ
- ▶ Modelo:

$$\hat{y} = f(x, \phi)$$

¿Cómo entrenamos esto?

¿Cómo entrenamos esto?

Necesitamos:

- ▶ un conjunto de entrenamiento: $\mathcal{D}_{train} = \{(x_i, y_i)\}_{i=1}^n$
- ▶ una función de pérdida por observación: $\ell(f(x_i), y_i)$
- ▶ una función de pérdida sobre el conjunto de entrenamiento:

$$L(\phi) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i, \phi), y_i)$$

¿Cómo entrenamos esto?

Buscamos los parámetros que minimicen la pérdida:

$$\hat{\phi} = \arg \min_{\phi} L(\phi)$$

- ▶ Entrenamos usando el conjunto de entrenamiento
- ▶ Seleccionamos hiperparámetros usando validación
- ▶ Evaluamos el modelo final sobre datos que no utilizamos durante el entrenamiento

Regresión lineal

¿Por qué regresamos a regresión lineal?

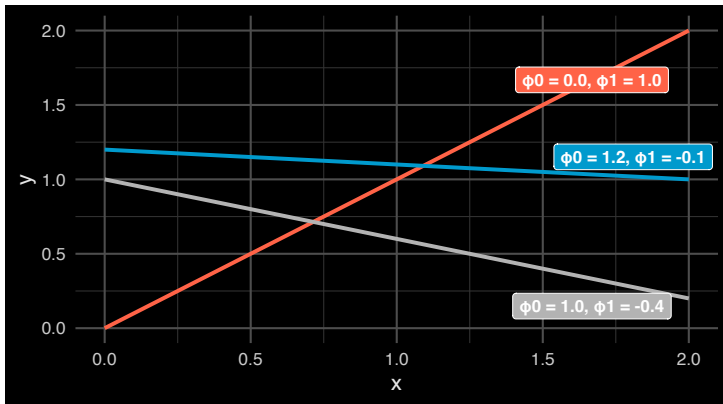
- ▶ Ya conocemos el algoritmo
- ▶ Podemos implementarlo desde cero
- ▶ Nos permite recordar los componentes básicos de ML
- ▶ Vamos a utilizarlo como puente hacia Deep Learning

La estructura que veremos aquí se mantiene:

modelo $\rightarrow$ loss $\rightarrow$ gradiente $\rightarrow$ optimización

Regresión lineal

- Modelo: $y = f(x, \phi) = \phi_0 + \phi_1 x$
- Parámetros: $\phi = [\phi_0 \phi_1]^T$



De regresión lineal a redes neuronales

En múltiples dimensiones:

$$f(x) = w^T x + b$$

Esta operación aparecerá constantemente en redes neuronales:

$$z = w^T x + b$$

Una red profunda combinará muchas transformaciones de este tipo con funciones no lineales.

Función de pérdida

Para regresión podemos usar pérdida cuadrática:

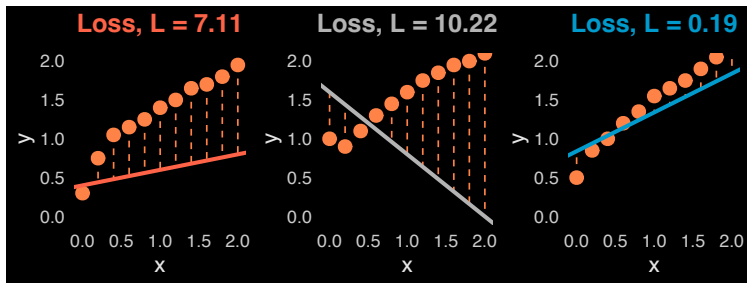
$$L(w, b) = \frac{1}{2n} \sum_{i=1}^n (w^T x_i + b - y_i)^2$$

Distintos parámetros producen distintas predicciones y distintas pérdidas.

Nuestro objetivo es:

$$\min_{w, b} L(w, b)$$

Regresión lineal



Optimización

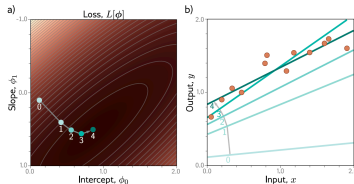
Gradient Descent

- ▶ Queremos encontrar parámetros que minimicen: $L(\phi)$
- ▶ El gradiente: $\nabla_{\phi} L(\phi)$ nos indica cómo cambia localmente la pérdida cuando modificamos los parámetros.

Gradient Descent actualiza:

$$\phi_{t+1} = \phi_t - \eta \nabla_{\phi} L(\phi_t)$$

donde η es la tasa de aprendizaje.



¿Por qué no usamos siempre todos los datos?

Con Gradient Descent usamos todas las observaciones:

$$\nabla L(\phi) = \frac{1}{n} \sum_{i=1}^n \nabla_{\phi} \ell_i(\phi)$$

antes de cada actualización.

Si n es grande, cada actualización puede ser costosa.

En el otro extremo, SGD utiliza una sola observación:

$$\nabla_{\phi} \ell_i(\phi)$$

Esto hace actualizaciones baratas, pero el gradiente puede tener mucha variabilidad y no siempre aprovecha eficientemente el hardware.

Mini-batch SGD

En la práctica utilizamos un subconjunto de observaciones:

$$B_t \subset \{1, \dots, n\}$$

En cada iteración:

1. sampleamos un mini-batch B_t
2. calculamos las predicciones
3. calculamos la pérdida
4. calculamos los gradientes
5. actualizamos los parámetros

$$\phi_{t+1} = \phi_t - \frac{\eta}{|B_t|} \sum_{i \in B_t} \nabla_{\phi} \ell_i(\phi_t)$$

¿Por qué mini-batches?

Mini-batch SGD busca un compromiso entre:

- ▶ costo por actualización
- ▶ variabilidad del gradiente
- ▶ paralelización en GPUs/TPUs
- ▶ uso de memoria

El batch size depende de:

- ▶ memoria y hardware disponible
- ▶ arquitectura del modelo
- ▶ tamaño de cada observación
- ▶ propiedades de la optimización

No existe un batch size universalmente óptimo.

Valores como 32, 64, 128 o 256 son comunes en algunos problemas.

Inicialización

Para regresión lineal podemos empezar, por ejemplo, con:

$$w = 0$$

$$b = 0$$

y Gradient Descent funciona correctamente.

En redes neuronales normalmente **no podemos inicializar todos los parámetros en cero.**

¿Por qué?

Lo veremos cuando estudiemos inicialización y simetrías.

¿Por qué métodos iterativos?

Para regresión lineal existe una solución analítica.

En ciertas condiciones:

$$\hat{w} = (X^T X)^{-1} X^T y$$

aunque en la práctica no solemos calcular explícitamente la inversa.

Podemos resolver el sistema utilizando métodos numéricos como:

- ▶ QR
- ▶ SVD
- ▶ Cholesky

Pero los modelos que veremos en Deep Learning generalmente **no tienen una solución analítica equivalente**.

¿Por qué no hacer búsqueda exhaustiva?

Supongamos que tenemos:

- ▶ d parámetros
- ▶ k posibles valores para cada parámetro

Entonces existen: k^d combinaciones.

Si tenemos 1000 parámetros y solo 10 valores posibles por parámetro: 10^{1000} combinaciones.

Además, normalmente nuestros parámetros son continuos.

El gradiente nos permite utilizar información sobre la geometría de la función de pérdida para decidir hacia dónde movernos.

Interpretación probabilística

¿De dónde viene la pérdida cuadrática?

Supongamos: $y_i = w^T x_i + b + \epsilon_i$

donde: $\epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$

Entonces:

$$y_i \mid x_i, w, b \sim N(w^T x_i + b, \sigma^2)$$

y:

$$p(y_i \mid x_i, w, b) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y_i - w^T x_i - b)^2}$$

Máxima verosimilitud y MSE

En lugar de maximizar la likelihood podemos minimizar la negative log-likelihood:

$$-\log p(\mathbf{y}|X, w, b) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i - b)^2$$

El primer término no depende de w ni de b .

Por lo tanto:

$$\arg \max_{w, b} p(\mathbf{y}|X, w, b) = \arg \min_{w, b} \sum_{i=1}^n (y_i - w^T x_i - b)^2$$

Minimizar la pérdida cuadrática es equivalente a máxima verosimilitud bajo un modelo con ruido gaussiano de varianza constante.

Esto aparecerá otra vez

Muchas funciones de pérdida pueden entenderse probabilísticamente.

Por ejemplo:

Gaussian $\implies$ MSE

Bernoulli $\implies$ Binary Cross-Entropy

Categorical $\implies$ Cross-Entropy

Generalización

¿Cómo sabemos si el modelo funciona?

Podemos medir su desempeño sobre entrenamiento:

$$\hat{R}_{train}(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i)$$

Este es nuestro **riesgo empírico** sobre el training set.

Pero este no es realmente nuestro objetivo.

Queremos que el modelo funcione sobre observaciones nuevas.

Riesgo de población

Idealmente queremos conocer:

$$R(f) = E_{(X,Y) \sim P}[\ell(f(X), Y)]$$

Equivalentemente:

$$R(f) = \int \int \ell(f(x), y) p(x, y) dx dy$$

Este es el riesgo de población.

El problema es que no conocemos exactamente:

$$p(x, y)$$

El problema de generalización

Nuestro modelo final depende del conjunto de entrenamiento:

$$\hat{f} = \mathcal{A}(\mathcal{D}_{train})$$

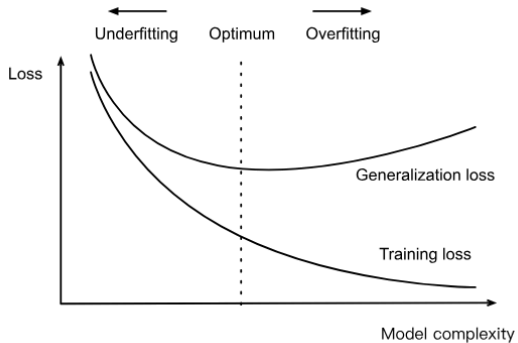
Por eso evaluar $\hat{f}$ sobre los mismos datos utilizados para construirlo puede producir una estimación demasiado optimista.

La pregunta central es:

¿Qué tan bien funciona $\hat{f}$ sobre nuevas observaciones?

Overfitting vs Underfitting

Overfitting vs Underfitting



Underfitting

El modelo no captura suficientemente bien la estructura útil de los datos.

Normalmente observamos:

$$R_{train} \text{ alto}$$

y:

$$R_{test} \text{ alto}$$

Overfitting

El modelo obtiene un desempeño muy bueno sobre entrenamiento, pero peor sobre datos nuevos.

Normalmente:

$$R_{train} \text{ bajo}$$

pero:

$$R_{test} > R_{train}$$

La diferencia entre ambos se suele llamar:

generalization gap

Más datos y generalización

En muchos problemas, aumentar el número de observaciones de entrenamiento mejora el desempeño fuera de muestra.

Intuitivamente: $n \uparrow$ nos da más información sobre la distribución de los datos.

Sin embargo, más datos no garantizan automáticamente un mejor modelo.

También importan:

- ▶ calidad de los datos
- ▶ representatividad
- ▶ distribución de entrenamiento y prueba
- ▶ arquitectura
- ▶ optimización
- ▶ regularización

Deep Learning complica un poco esta historia

La intuición clásica dice:

modelos demasiado complejos tienden a overfittear.

Pero las redes neuronales modernas frecuentemente tienen: \$ \text{número de parámetros} \gg \text{número de observaciones} \$

y alcanzan en muchos casos: \$ R_{\text{train}} \approx 0 \$

sin necesariamente generalizar mal.

Por lo tanto (en deep learning!):

más parámetros $\nRightarrow$ automáticamente más overfitting

Volveremos a esta pregunta durante el curso.

Selección de modelos

Train, Validation y Test

Tenemos tres tareas diferentes:

Training

Aprender los parámetros del modelo: $w, b, \dots$

Validation

Seleccionar hiperparámetros y modelos:

η, λ , batch size, arquitectura, ...

Test

Estimar el desempeño final sobre datos que no participaron en nuestras decisiones.

Train, Validation y Test

Podemos pensar en el proceso como:

$$\mathcal{D}_{train} \longrightarrow \text{parámetros}$$

$$\mathcal{D}_{validation} \longrightarrow \text{hiperparámetros / modelo}$$

$$\mathcal{D}_{test} \longrightarrow \text{evaluación final}$$

No debemos utilizar repetidamente el conjunto de test para tomar decisiones sobre el modelo.

Si hacemos eso, empezamos indirectamente a sobreajustarnos al test set.

Regularización

Regularización

Una manera de modificar nuestro problema de entrenamiento es añadir una penalización.

Por ejemplo, regularización L_2 :

$$L_{reg}(w, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (w^T x_i + b - y_i)^2 + \frac{\lambda}{2} \|w\|_2^2$$

donde:

$$\lambda \geq 0$$

controla la fuerza de la regularización.

¿Qué hace L_2 ?

L_2 penaliza pesos de magnitud grande:

$$\frac{\lambda}{2} \|w\|_2^2 = \frac{\lambda}{2} \sum_j w_j^2$$

Esto tiende a producir pesos de menor magnitud.

Normalmente no regularizamos el intercepto b .

L_2 y SGD

Sabemos que:

$$\nabla_w \frac{\lambda}{2} \|w\|_2^2 = \lambda w$$

Entonces la actualización usando mini-batch SGD es:

$$w \leftarrow w - \eta \left[\frac{1}{|B|} \sum_{i \in B} x_i (w^T x_i + b - y_i) + \lambda w \right]$$

Weight Decay

Podemos reescribir:

$$w \leftarrow (1 - \eta\lambda)w - \frac{\eta}{|B|} \sum_{i \in B} x_i (w^T x_i + b - y_i)$$

El término:

$$(1 - \eta\lambda)w$$

reduce ligeramente la magnitud de los pesos en cada actualización.

De aquí viene el nombre:

weight decay

¿Por qué necesitamos regularización?

PRÓXIMAMENTE.

Pero esta pregunta se vuelve especialmente importante cuando tenemos modelos con:

$$10^6, \quad 10^9, \quad 10^{12}$$

parámetros.

De Machine Learning a Deep Learning

¿Qué permanece igual?

Hasta ahora tenemos:

$$x \rightarrow f_{\phi}(x) \rightarrow L(\phi) \rightarrow \nabla_{\phi} L \rightarrow \text{actualización de parámetros}$$

Esta estructura **no cambia** en Deep Learning.

¿Qué cambia?

En regresión lineal:

$$f(x) = w^T x + b$$

En Deep Learning tendremos composiciones:

$$f_{\phi}(x) = f^{(L)} \left(f^{(L-1)} \left(\dots f^{(1)}(x) \right) \right)$$

Ahora nuestros modelos pueden tener millones o miles de millones de parámetros.